

TalentLock

Security Review & Remediation Report

Deep code audit — API server, storage, auth & repo hygiene

Prepared: 5 July 2026 · Scope: `artifacts/api-server`, `artifacts/talentlock`, repository configuration

Result: 7 security issues identified and fixed — **2 Critical** **2 High** **3 Medium**. All fixes verified with a passing TypeScript compile of the API server.

1. Executive Summary

A deep, file-by-file security review was performed across the TalentLock backend (Express 5 API, object storage, authentication and authorization helpers), the React frontend, and the repository configuration. The review focused on access control (IDOR), authentication weaknesses, injection, secrets management, cross-origin policy, and information disclosure.

Seven issues were found and remediated. The two most serious were an **unauthenticated path to private signed-contract PDFs** and a **hardcoded admin signing secret and password** that together allowed full admin impersonation. The existing per-resource authorization layer, SQL parameterization, input sanitisation and secret-redaction were reviewed and found to be sound — those areas required no change.

Fields reviewed: Broken access control / IDOR · Authentication & session integrity · Credential & secret management · Cross-Origin Resource Sharing (CORS) · SQL / query injection · Cross-site scripting (XSS) · CSRF · File upload handling · Client-IP trust & rate limiting · Prompt/AI input handling · Repository hygiene & secret leakage · GDPR / PII handling.

2. Findings Overview

#	Severity	Issue	Area	Status
1	CRITICAL	Unauthenticated access to private stored objects (signed-contract PDFs) via the object-serving route	Access control / IDOR	Fixed
2	CRITICAL	Hardcoded admin session-signing secret & default admin password (forgeable admin session / public backdoor)	Auth / secrets	Fixed
3	HIGH	Forgeable local storage signed URLs (same in-source secret)	Secrets / storage	Fixed
4	HIGH	Overly permissive CORS: any origin reflected with credentials enabled	CORS	Fixed
5	MEDIUM	Demo login enabled by default in shipped example config	Auth / config	Fixed
6	MEDIUM	Client-IP not trusted behind proxy — rate limiting & audit IPs unreliable	Rate limiting / audit	Fixed
7	MEDIUM	Internal security report & build logs committed to version control	Info disclosure	Fixed

3. Detailed Findings & Fixes

Finding 1 — Unauthenticated access to private objects (IDOR) CRITICAL

File: artifacts/api-server/src/routes/storage.ts — GET /api/storage/objects/*path

What was wrong: The route only enforced authorization for two path prefixes: documents/ (admin only) and uploads/<userId>/ (owner only). Any other private key fell straight through to the storage read with **no authentication or ownership check**. Fully-signed legal agreements are cached at agreements/<id>/signed-agreement.pdf, so any visitor could enumerate agreement IDs and download other parties' executed contracts — bypassing the participant check that guards the dedicated /agreements/:id/download route.

Impact: Disclosure of confidential signed contracts (names, rates, terms, signatures) belonging to any user; broad IDOR over any private object not under the two whitelisted prefixes.

Fix: Converted the route to **deny-by-default**. A request must match a known, access-controlled prefix (documents/ admin-gated, or uploads/<userId>/ owner-gated); every other path now returns 403 Forbidden. Private contract PDFs remain reachable only through their participant-gated download route.

Finding 2 — Hardcoded admin secret & default password **CRITICAL**

File: artifacts/api-server/src/lib/adminAuth.ts

What was wrong: When `SESSION_SECRET` was unset the admin session cookie (`tl_admin`) was HMAC-signed with a constant baked into source (`talentlock-dev-secret-do-not-use-in-prod`). Anyone with the source could forge a valid admin cookie and gain full admin-console access. Separately, when `ADMIN_PASSWORD` was unset the login fell back to a well-known default password (`TalentLockAdmin123!`) — a public backdoor. Both fallbacks emitted only a log warning and otherwise worked normally.

Impact: Complete administrative takeover: user data export, bulk data deletion (`/admin/clear-transactions`, `/admin/reset-test-data`), document review queue, audit logs.

Fix: Removed both hardcoded values. If `SESSION_SECRET` is missing, cookies are now signed with a **random per-process secret** (unforgeable; sessions simply don't persist across restarts). If `ADMIN_PASSWORD` is missing, admin login **fails closed** (disabled) instead of accepting a default. Credential comparison remains timing-safe.

Finding 3 — Forgeable local storage signed URLs **HIGH**

File: artifacts/api-server/src/lib/localObjectStorage.ts

What was wrong: Local upload/read pre-signed URLs were HMAC-signed with the same in-source constant secret. An attacker knowing the secret (public in the repo) could forge signed `PUT` / `GET` URLs to write or read arbitrary keys within the storage root, defeating the URL-expiry and integrity guarantees.

Impact: Unauthorised upload/overwrite and read of stored objects when local object storage is in use.

Fix: Replaced the constant with a **random per-process fallback secret** when `SESSION_SECRET` is unset. Existing path-traversal protection (rejecting `..` / absolute paths) was verified and retained.

Finding 4 — Overly permissive CORS **HIGH**

File: artifacts/api-server/src/app.ts

What was wrong: `cors({ credentials: true, origin: true })` reflects the caller's `Origin` back in `Access-Control-Allow-Origin` for every site while also allowing credentials. This lets any malicious website issue authenticated cross-origin requests against the API on a logged-in victim's behalf and read the responses.

Impact: Cross-origin data theft and state-changing actions using the victim's session.

Fix: Restricted to an explicit allowlist from `ALLOWED_ORIGINS` (falling back to `APP_URL`). Same-origin / non-browser requests (no `Origin`) are permitted; unknown origins are rejected. A wildcard is allowed only in non-production when no allowlist is configured, to preserve local-dev convenience.

Finding 5 — Demo login enabled by default in example config **MEDIUM**

File: .env.example · artifacts/api-server/src/routes/demo.ts

What was wrong: `POST /api/demo/sign-in-token` mints Clerk sign-in tokens for shared, hardcoded demo accounts. It is allowed in production whenever `ENABLE_DEMO_LOGIN=true` — and the shipped `.env.example` set exactly that. Copying the example to production would let anyone authenticate as the demo employer/freelancer.

Impact: Anonymous account takeover of the demo identities in a misconfigured production deployment.

Fix: Changed the example default to `ENABLE_DEMO_LOGIN=false` with an explicit "never enable in production" warning. Also documented `SESSION_SECRET`, `ADMIN_PASSWORD`, `ALLOWED_ORIGINS` and `TRUST_PROXY` requirements in the example.

Finding 6 — Client IP not trusted behind proxy MEDIUM

File: artifacts/api-server/src/app.ts

What was wrong: `trust proxy` was left disabled. Behind a reverse proxy (Railway/Render/nginx) `req.ip` becomes the proxy's address for all requests. This collapses the admin-login rate limiter (all attempts share one bucket, so it is trivially bypassable or falsely locks everyone out) and records the wrong IP in the audit log.

Impact: Ineffective brute-force protection on admin login; inaccurate security audit trail.

Fix: Added a `TRUST_PROXY` environment toggle (hop count or `true`) so the real client IP is read from `X-Forwarded-For` in proxied deployments, restoring correct rate-limiting and audit attribution.

Finding 7 — Sensitive files committed to version control MEDIUM

Files: `_sec.txt`, `.tmp-api.log`, `.tmp-install.log`, `.tmp-ui.log`

What was wrong: An internal security/production-readiness report (which enumerates architecture and prior vulnerabilities) and local build/run logs were tracked in git. This is information disclosure that hands an attacker a roadmap and leaks internal environment detail.

Impact: Disclosure of internal security posture and environment details to anyone with repository access.

Fix: Untracked the files from git (`git rm --cached` , files retained locally) and added `.tmp-*.log` and `_sec.txt` to `.gitignore` to prevent recurrence.

4. Areas Reviewed — No Change Required

These controls were audited and found to be implemented correctly; they are recorded here as positive assurance.

Control	Assessment
SQL / query injection	All database access uses Drizzle's query builder or parameterized <code>sql</code> template tags; the full-text search input passes through <code>sanitizeSearchQuery()</code> . No string-concatenated SQL found.
Per-resource authorization	<code>accessControl.ts</code> enforces participant/owner checks on bookings, meetings, agreements, conversations and job requirements with a consistent 401 / 403 / 404 convention.
Agreement signing & download	Signer role is derived server-side (client-supplied role rejected on mismatch); download requires <code>status = fully_signed</code> and participant access.
Admin state-changing routes	Guarded by <code>requireAdmin</code> plus a <code>csrf-csrf</code> double-submit token on non-GET methods.
Cross-site scripting (XSS)	React auto-escaping throughout. The only <code>dangerouslySetInnerHTML</code> is the shadcn chart CSS block driven by developer-defined colour config, not user input.
Input sanitisation	Free-text writes pass through <code>sanitizeText()</code> / <code>sanitizeRichText()</code> (HTML stripped; links limited to <code>https</code>).
Secret redaction in logs	Pino redacts authorization/cookie headers, passwords, tokens, signature URLs and clerk IDs.
File uploads	Resume and document uploads validate MIME type and enforce size limits; document reads are admin-gated.
GDPR / PII	Account deletion anonymises data in a transaction before calling Clerk, with an active-booking guard and retry-on-failure handling.

Note (by design, not a vulnerability): `POST /api/subscriptions/upgrade` is a simulated checkout with no real payment step, as specified for the current milestone. It should be wired to the real payment provider before handling live billing.

5. Recommended Operational Hardening

- Set strong, unique values for `SESSION_SECRET`, `ADMIN_PASSWORD` and `CSRF_SECRET` in every non-local environment (the app now fails closed / uses ephemeral secrets without them).
- Populate `ALLOWED_ORIGINS` and `TRUST_PROXY` in production.
- Keep `ENABLE_DEMO_LOGIN` unset/false in production.
- Consider a general API rate limiter (not just admin login) and rotating any secrets that may have been exposed via the previously committed files.

6. Verification

All code changes were validated with `pnpm --filter @workspace/api-server run typecheck`, which completed with no errors. Fixes are additive and preserve existing legitimate behaviour (owner-scoped uploads, admin document access, local-dev CORS and storage).